APPLICATIONS WEB DISTRIBUÉES · PROJET AWD

MediConnect

Plateforme intelligente de coordination des services médicaux

Dossier technique — Conception microservices

Validation intermédiaire 1 · Séance 4 · Année universitaire 2026-2027

ÉQUIPE

Rawen Aouina
Sarra Mejbri
Bochra Damak
Rania Ben Haj Hmida
Bechir Belgaied
Yassine Gasmi

Sommaire

1. Description du sujet
2. Acteurs et fonctionnalités
3. Découpage en microservices
4. Données gérées par chaque service
5. Interactions entre les services
6. Diagramme d'architecture globale
7. Autres éléments nécessaires à la compréhension de la conception

Ce dossier décrit la conception de MediConnect présentée en séance 4, avant le développement métier. Chaque diagramme est également fourni dans son propre fichier PDF (dossier Documentation/Diagrams).

1. Description du sujet

1.1 Contexte

Le secteur de la santé connaît une transformation numérique qui facilite l'accès aux services médicaux. Pourtant, ces services restent **dispersés** entre établissements, pharmacies et plateformes, et la **coordination** entre les acteurs de santé reste difficile : le patient doit passer d'un outil à l'autre pour trouver un médicament, prendre rendez-vous, suivre un traitement ou consulter son carnet vaccinal.

1.2 Problématique

> Comment centraliser les différents services médicaux dans une seule plateforme afin de faciliter leur accès et d'améliorer la coordination entre les acteurs de santé ?

1.3 Objectifs

- Centraliser les services médicaux dans une plateforme unique.
- Faciliter l'accès aux médicaments (disponibilité, réservation, pharmacies de garde).
- Simplifier l'organisation des rendez-vous médicaux.
- Améliorer le suivi des traitements, des vaccinations et des dons de sang.
- Offrir aux administrateurs une vue globale grâce à des indicateurs.

1.4 Solution proposée

MediConnect est une plateforme web composée d'un frontend Angular et de sept microservices indépendants, chacun responsable d'une capacité métier et propriétaire de sa propre base MySQL. Les services sont accessibles par une API Gateway unique, sécurisés par Keycloak, découverts dynamiquement par Eureka, configurés par un Config Server et reliés entre eux par des appels REST (OpenFeign) ou des événements (RabbitMQ). L'ensemble est déployé avec Docker Compose.

> MediConnect facilite l'accès et la coordination des services ; les décisions médicales restent du ressort des professionnels de santé.

2. Acteurs et fonctionnalités

2.1 Acteurs

Tableau 1 — Acteurs de la plateforme

Acteur	Description	Besoins principaux
Patient	Utilisateur principal de la plateforme	Trouver et réserver un médicament, prendre rendez-vous, suivre son traitement, consulter son carnet vaccinal, trouver une garde
Médecin	Professionnel de santé	Gérer ses créneaux et consultations, prescrire un traitement, enregistrer une vaccination, publier une demande de sang
Pharmacien	Responsable d'une pharmacie	Gérer les stocks et les réservations, déclarer les gardes
Donneur	Personne inscrite au don de sang	S'inscrire, consulter les demandes, enregistrer un don
Administrateur	Supervise la plateforme	Consulter le tableau de bord et les rapports, gérer les comptes

2.2 Cas d'utilisation général

Le diagramme suivant présente les douze cas d'utilisation principaux. Tous sont accessibles après authentification auprès de Keycloak.

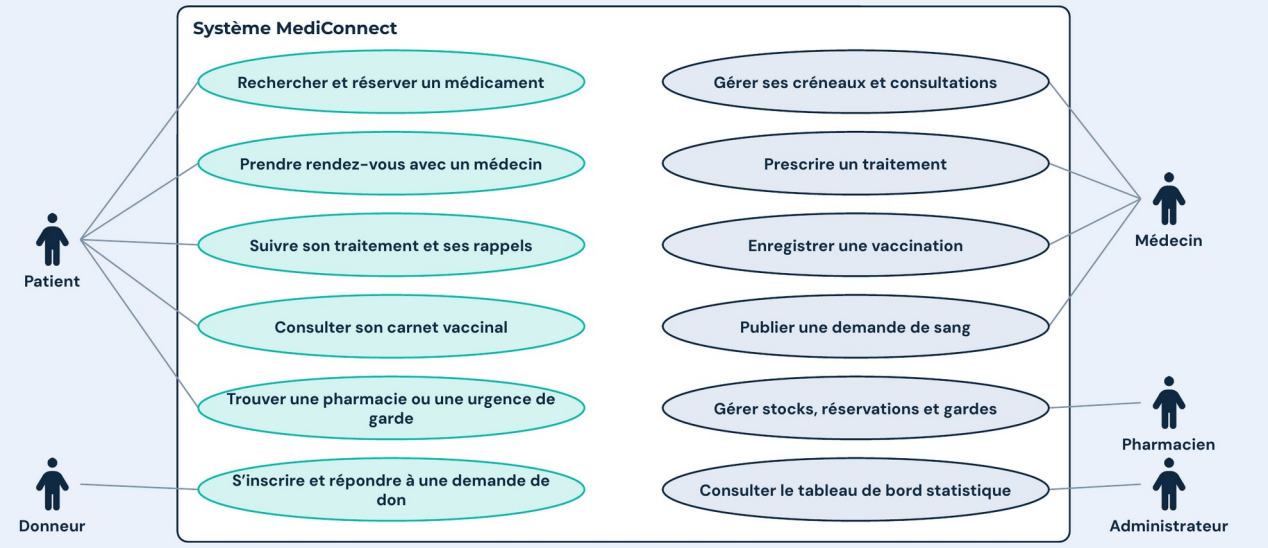


Figure 1 — Diagramme de cas d'utilisation général (fichier : 01-cas-utilisation-general.pdf)

Tableau 2 — Cas d'utilisation et microservice responsable

Cas d'utilisation	Acteur	Microservice
Rechercher et réserver un médicament	Patient	MS1
Prendre rendez-vous avec un médecin	Patient	MS3
Suivre son traitement et ses rappels	Patient	MS4
Consulter son carnet vaccinal	Patient	MS6
Trouver une pharmacie ou une urgence de garde	Patient	MS5 (+ MS1)
S'inscrire et répondre à une demande de don	Donneur	MS2
Gérer ses créneaux et consultations	Médecin	MS3

Cas d'utilisation	Acteur	Microservice
Prescrire un traitement	Médecin	MS4 (+ MS1)
Enregistrer une vaccination	Médecin	MS6
Publier une demande de sang	Médecin	MS2
Gérer stocks, réservations et gardes	Pharmacien	MS1, MS5
Consulter le tableau de bord statistique	Administrateur	MS7

2.3 Principales fonctionnalités

- Réserver un médicament en pharmacie après avoir vérifié sa disponibilité.
- Publier et suivre les demandes de don de sang.
- Prendre, consulter et annuler un rendez-vous avec un médecin.
- Recevoir les rappels de prise de traitement, de rendez-vous et de vaccin.
- Trouver une pharmacie de garde ou un service d'urgence par zone.
- Tenir son carnet de vaccination et connaître ses prochaines échéances.
- Suivre les indicateurs globaux de la plateforme (administrateur).

2.4 Périmètre

Le périmètre est volontairement réaliste pour une équipe de six : livrer d'abord le flux métier complet de chaque service, puis enrichir l'expérience.

Tableau 3 — Périmètre fonctionnel

Inclus dans le projet	Hors périmètre / limites
Médicaments, stocks et réservations	Diagnostic médical automatisé
Dons de sang	Prescription médicale autonome
Rendez-vous médicaux	Paiement en ligne
Traitements et rappels	Téléconsultation (non prévue à ce stade)
Gardes et urgences	
Carnet de vaccination	
Analyse et statistiques (service commun)	

3. Découpage en microservices

3.1 Principe de découpage

Le découpage suit les **capacités métier** (contextes bornés) et non les tables ou les écrans : chaque microservice correspond à un domaine qu'un membre de l'équipe peut développer, tester et déployer indépendamment. Chaque service possède ses données ; aucun service n'accède directement à la base d'un autre.

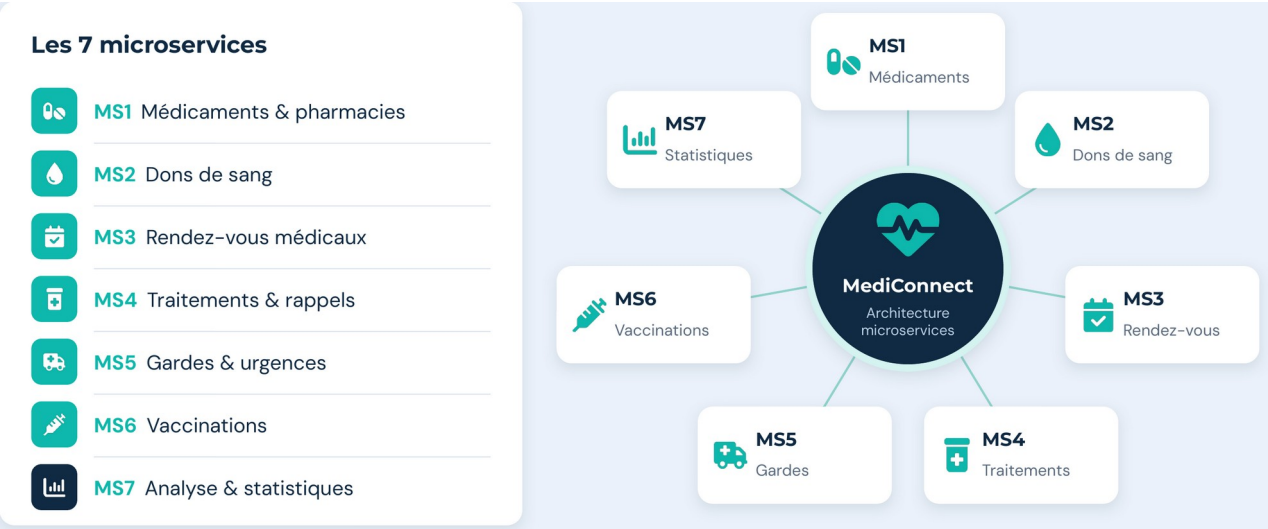


Figure 2 — Découpage de MediConnect en sept microservices (fichier : 02-decoupage-microservices.pdf)

3.2 Les sept microservices

Tableau 4 — Microservices de MediConnect

Service	Capacité métier	Technologie	Base	Port
MS1 Médicaments & pharmacies	Catalogue des médicaments, stocks des pharmacies, réservations	Spring Boot	MySQL `ms1_medicaments`	8081
MS2 Dons de sang	Donneurs, demandes de sang et dons	Spring Boot	MySQL `ms2_dons_sang`	8082
MS3 Rendez-vous médicaux	Médecins, créneaux et rendez-vous	Spring Boot	MySQL `ms3_rendez_vous`	8083
MS4 Traitements & rappels	Traitements prescrits, prises et rappels	Spring Boot	MySQL `ms4_traitements`	8084
MS5 Gardes & urgences	Pharmacies de garde et services d'urgence	Spring Boot	MySQL `ms5_gardes`	8085
MS6 Vaccinations	Carnet vaccinal, doses et échéances	Spring Boot	MySQL `ms6_vaccinations`	8086
MS7 Analyse & statistiques (commun)	Indicateurs et statistiques de la plateforme	Python · FastAPI	MySQL `ms7_statistiques`	8087

3.3 Responsabilités et absence de recouvrement

MS1 — Médicaments & pharmacies

- Gérer le catalogue des médicaments

- Suivre la disponibilité par pharmacie (stocks)
- Gérer les réservations de médicaments
- Servir de référentiel des pharmacies pour MS5

MS2 — Dons de sang

- Inscrire les donneurs et leur groupe sanguin
- Publier et suivre les demandes de sang
- Enregistrer les dons et l'historique du donneur

MS3 — Rendez-vous médicaux

- Gérer les médecins et leurs créneaux
- Permettre la prise et l'annulation de rendez-vous
- Conserver l'historique des consultations

MS4 — Traitements & rappels

- Créer les traitements et leurs prescriptions
- Planifier les prises
- Envoyer les rappels (prises, rendez-vous, vaccins)
- Conserver l'historique du suivi

MS5 — Gardes & urgences

- Publier le planning des pharmacies de garde
- Référencer les services d'urgence
- Rechercher par zone et par date

MS6 — Vaccinations

- Tenir le carnet vaccinal numérique
- Enregistrer les doses administrées
- Calculer les prochaines échéances (les rappels sont envoyés par MS4)

MS7 — Analyse & statistiques (commun)

- Consommer les événements publiés par les autres services
- Calculer des indicateurs agrégés par période
- Exposer le tableau de bord et les rapports à l'administrateur

Deux choix évitent que deux services gèrent la même donnée :

- **Pharmacies** : MS1 est le référentiel des pharmacies ; MS5 ne stocke que `pharmacieId` et récupère le nom et l'adresse auprès de MS1.
- **Rappels** : MS4 envoie tous les rappels (prises, rendez-vous, vaccins) ; MS3 et MS6 publient des événements que MS4 consomme.

3.4 Microservice commun et technologie avancée

MS7 — Analyse et statistiques est développé par toute l'équipe et présenté par un seul membre. Il utilise **Python et FastAPI** : c'est la technologie avancée du projet. Ce choix montre l'interopérabilité de l'architecture (un service non-Java s'enregistre dans Eureka et est routé par la Gateway) et s'appuie sur l'écosystème Python, adapté au traitement et à l'agrégation de données.

4. Données gérées par chaque service

4.1 Une base MySQL par microservice

Chaque microservice possède **sa propre base MySQL** et son propre utilisateur, qui n'a de droits que sur cette base. Aucune base n'est partagée : un service qui a besoin d'une donnée d'un autre service passe par son API (synchrone) ou par un événement (asynchrone). Les références entre services se font uniquement par identifiant (par exemple `medicamentId` dans MS4, `pharmacieId` dans MS5).

Pourquoi MySQL pour tous les services ? Les données de MediConnect sont relationnelles (entités liées par des clés étrangères) et plusieurs opérations exigent des transactions ACID (stock et réservation, unicité d'un créneau). Une technologie unique simplifie aussi l'installation, Docker Compose et la montée en compétence de l'équipe.

Tableau 5 — Bases de données par service

Service	Base	Utilisateur	Justification du choix
MS1	`ms1_medicaments`	`ms1_user`	MySQL garantit des transactions ACID : le stock et la réservation sont modifiés ensemble, ce qui évite toute sur-réservation.
MS2	`ms2_dons_sang`	`ms2_user`	Les relations donneur → dons ← demandes et les contraintes d'intégrité (clés étrangères, unicité) sont naturellement prises en charge par une base relationnelle.
MS3	`ms3_rendez_vous`	`ms3_user`	La contrainte d'unicité sur le créneau, garantie par MySQL, empêche deux réservations simultanées du même créneau.
MS4	`ms4_traitements`	`ms4_user`	Les données sont fortement relationnelles (traitement → prescriptions → prises) : MySQL est adapté.
MS5	`ms5_gardes`	`ms5_user`	Les recherches par date et par zone s'appuient sur des index MySQL.
MS6	`ms6_vaccinations`	`ms6_user`	L'historique vaccinal doit être fiable et cohérent (carnet → doses → vaccins) : MySQL et ses contraintes conviennent.
MS7	`ms7_statistiques`	`ms7_user`	Les indicateurs sont calculés par des agrégations SQL (GROUP BY par période) ; MySQL est accessible depuis Python via SQLAlchemy.

4.2 MS1 — Médicaments & pharmacies

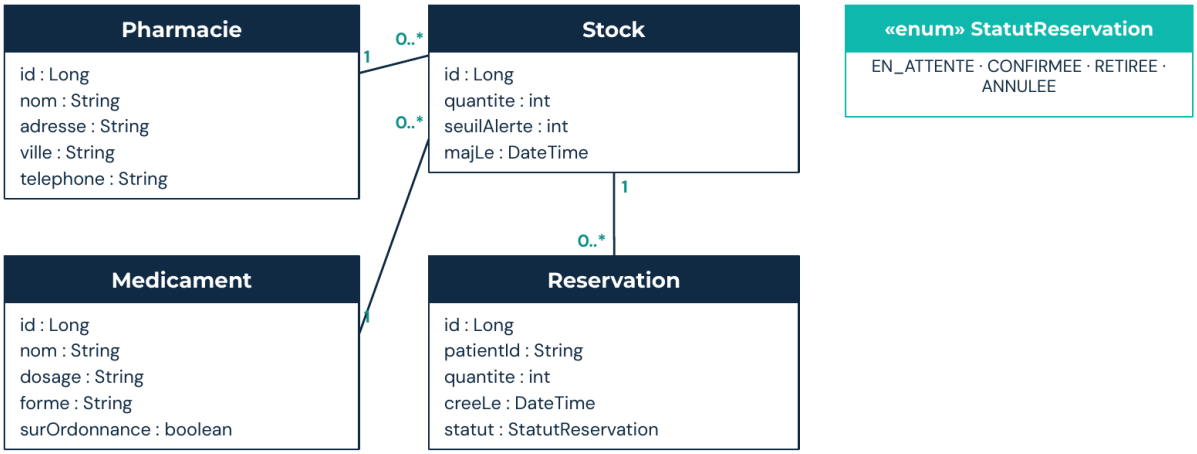


Figure 3 — Diagramme de classes de MS1 (fichier : 04-classes-ms1-medicaments.pdf)

Entité	Attributs principaux	Rôle
Pharmacie	id, nom, adresse, ville, telephone	Officine référencée (référentiel partagé par identifiant)
Medicament	id, nom, dosage, forme, surOrdonnance	Fiche d'un médicament
Stock	id, quantite, seuilAlerte, majLe	Quantité d'un médicament dans une pharmacie
Reservation	id, patientId, quantite, creeLe, statut	Réservation d'un patient sur un stock

Relations. Une pharmacie possède plusieurs stocks (1 → 0..*) ; un médicament est présent dans plusieurs stocks (1 → 0..*) ; un stock peut faire l'objet de plusieurs réservations (1 → 0..*).

Énumérations. StatutReservation : EN_ATTENTE, CONFIRMEE, RETIREE, ANNULEE.

Règles de gestion.

- Une réservation ne peut pas dépasser la quantité disponible (sinon 409).
- La mise à jour du stock et la création de la réservation se font dans la même transaction.

4.3 MS2 — Dons de sang

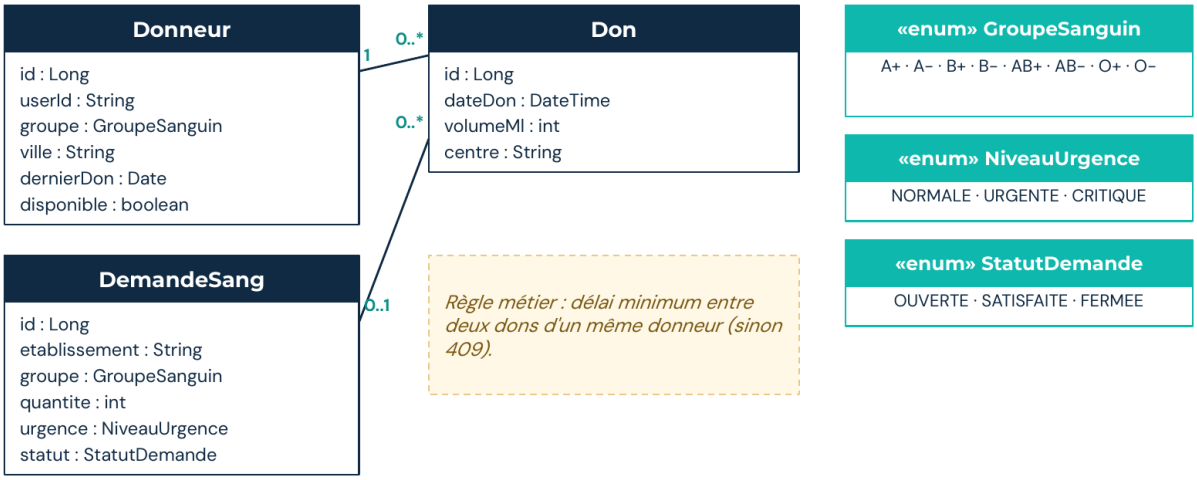


Figure 4 — Diagramme de classes de MS2 (fichier : 05-classes-ms2-dons-sang.pdf)

Entité	Attributs principaux	Rôle
Donneur	id, userId, groupe, ville, dernierDon, disponible	Personne inscrite comme donneur
DemandeSang	id, etablissement, groupe, quantite, urgence, statut	Besoin exprimé par un établissement / médecin
Don	id, dateDon, volumeMl, centre	Don effectué, éventuellement lié à une demande

Relations. Un donneur réalise plusieurs dons (1 → 0..*) ; une demande peut être satisfaite par plusieurs dons (0..1 → 0..*).

Énumérations. GroupeSanguin : A+, A−, B+, B−, AB+, AB−, O+, O− ; NiveauUrgence : NORMALE, URGENTE, CRITIQUE ; StatutDemande : OUVERTE, SATISFAITE, FERMEE.

Règles de gestion.

- Un délai minimum doit séparer deux dons d'un même donneur (sinon 409).
- Un donneur ne peut être inscrit qu'une fois (sinon 409).

4.4 MS3 — Rendez-vous médicaux

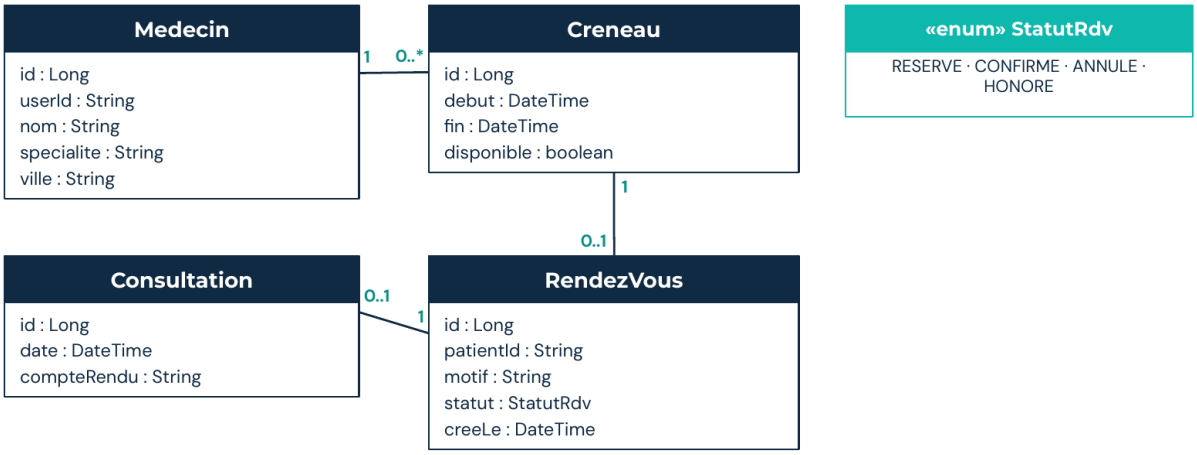


Figure 5 — Diagramme de classes de MS3 (fichier : 06-classes-ms3-rendez-vous.pdf)

Entité	Attributs principaux	Rôle
Medecin	id, userId, nom, specialite, ville	Médecin disponible sur la plateforme
Creneau	id, debut, fin, disponible	Plage horaire proposée par un médecin
RendezVous	id, patientId, motif, statut, creeLe	Réservation d'un créneau par un patient
Consultation	id, date, compteRendu	Trace de la consultation après le rendez-vous

Relations. Un médecin propose plusieurs créneaux (1 → 0..*) ; un créneau porte au plus un rendez-vous (1 → 0..1) ; un rendez-vous donne lieu à au plus une consultation (1 → 0..1).

Énumérations. StatutRdv : RESERVE, CONFIRME, ANNULE, HONORE.

Règles de gestion.

- Un créneau ne peut être réservé qu'une seule fois (contrainte d'unicité, sinon 409).
- Un rendez-vous annulé ou passé ne peut plus être annulé (409).

4.5 MS4 — Traitements & rappels

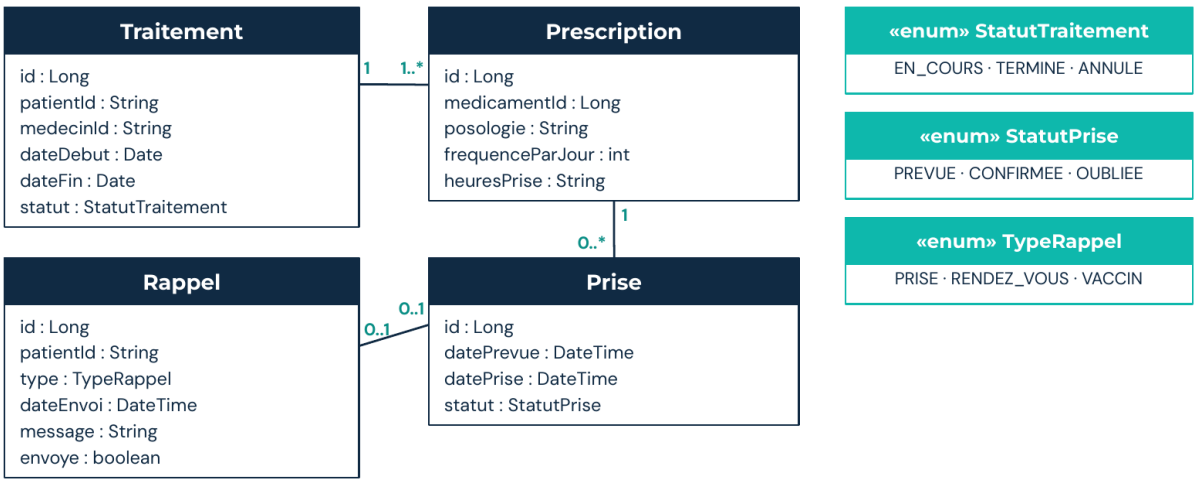


Figure 6 — Diagramme de classes de MS4 (fichier : 07-classes-ms4-traitements.pdf)

Entité	Attributs principaux	Rôle
Traitement	id, patientId, medecinId, dateDebut, dateFin, statut	Traitement prescrit à un patient
Prescription	id, medicamentId, posologie, frequenceParJour, heuresPrise	Ligne de traitement (médicament de MS1 par identifiant)
Prise	id, datePrevue, datePrise, statut	Prise planifiée puis confirmée ou oubliée
Rappel	id, patientId, type, dateEnvoi, message, envoye	Notification envoyée au patient

Relations. Un traitement contient une ou plusieurs prescriptions (1 → 1..*) ; une prescription génère plusieurs prises (1 → 0..*) ; une prise peut être associée à un rappel (0..1 — 0..1).

Énumérations. StatutTraitement : EN_COURS, TERMINE, ANNULE ; StatutPrise : PREVUE, CONFIRMEE, OUBLIEE ; TypeRappel : PRISE, RENDEZ_VOUS, VACCIN.

Règles de gestion.

- Une prise déjà confirmée ne peut pas être modifiée (409).
- Le médicament est référencé par son identifiant MS1 : il n'est pas recopié.

4.6 MS5 — Gardes & urgences

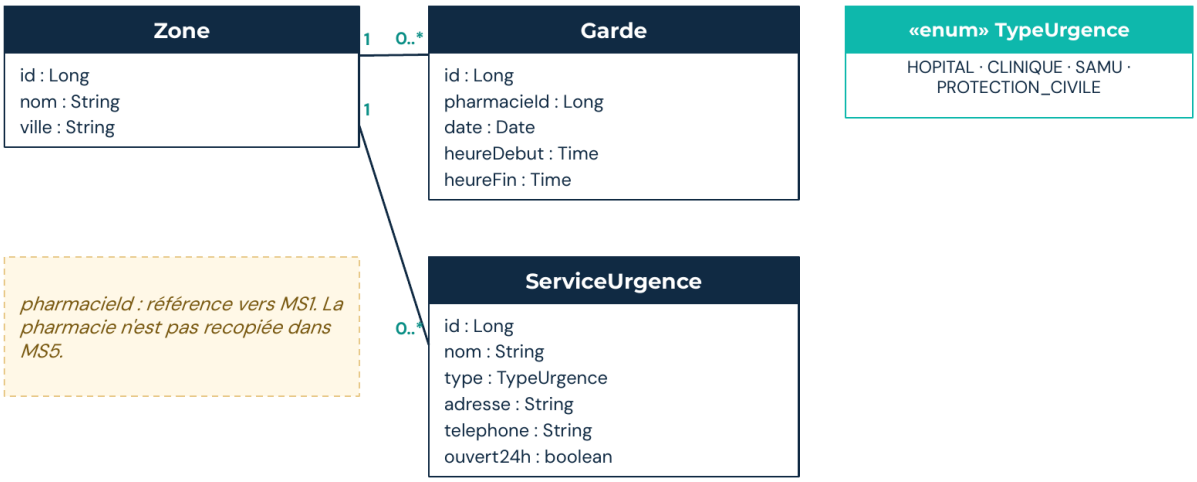


Figure 7 — Diagramme de classes de MS5 (fichier : 08-classes-ms5-gardes.pdf)

Entité	Attributs principaux	Rôle
Zone	id, nom, ville	Zone géographique de recherche
Garde	id, pharmacieId, date, heureDebut, heureFin	Garde d'une pharmacie (référence MS1)
ServiceUrgence	id, nom, type, adresse, telephone, ouvert24h	Hôpital, clinique, SAMU, protection civile

Relations. Une zone regroupe plusieurs gardes (1 → 0..*) et plusieurs services d'urgence (1 → 0..*). La pharmacie n'est pas recopiée : `pharmacieId` référence MS1.

Énumérations. TypeUrgence : HOPITAL, CLINIQUE, SAMU, PROTECTION_CIVILE.

Règles de gestion.

- Une pharmacie ne peut pas avoir deux gardes à la même date (409).

4.7 MS6 — Vaccinations

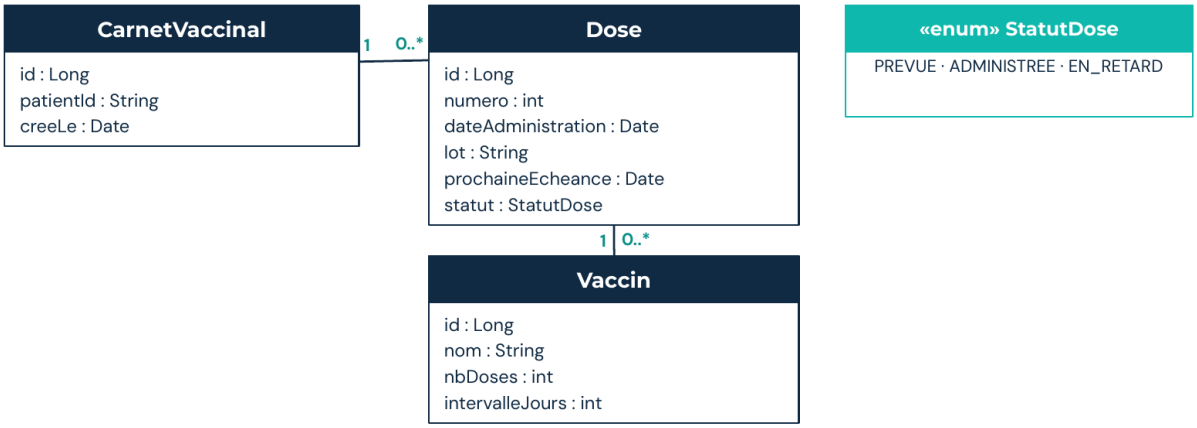


Figure 8 — Diagramme de classes de MS6 (fichier : 09-classes-ms6-vaccinations.pdf)

Entité	Attributs principaux	Rôle
CarnetVaccinal	id, patientId, creeLe	Carnet d'un patient
Vaccin	id, nom, nbDoses, intervalleJours	Référentiel des vaccins
Dose	id, numero, dateAdministration, lot, prochaineEcheance, statut	Dose administrée ou prévue

Relations. Un carnet contient plusieurs doses (1 → 0..*) ; chaque dose concerne un vaccin (0..* → 1).

Énumérations. StatutDose : PREVUE, ADMINISTREE, EN_RETARD.

Règles de gestion.

- Une même dose ne peut pas être enregistrée deux fois (409).
- La prochaine échéance est calculée à partir de l'intervalle du vaccin.

4.8 MS7 — Analyse & statistiques (commun)

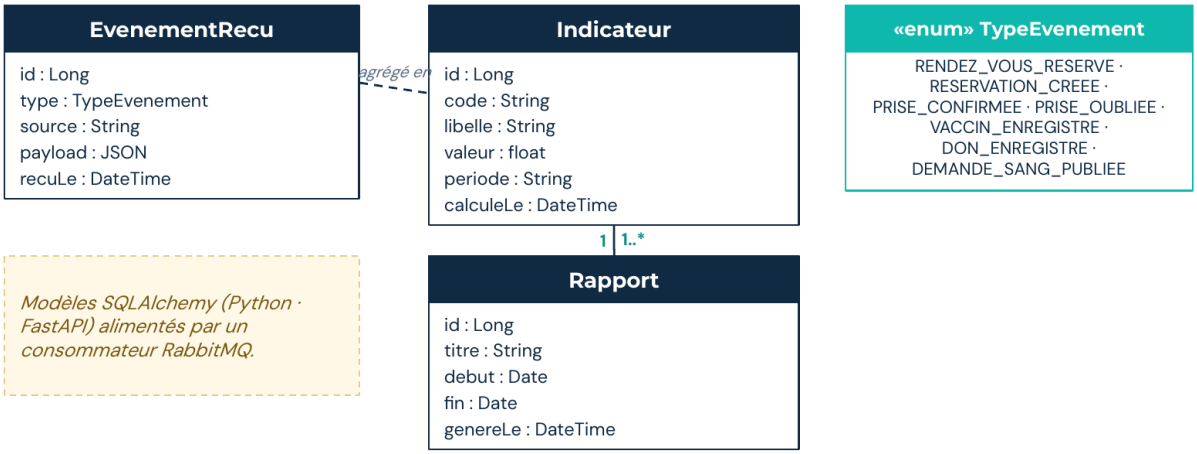


Figure 9 — Diagramme de classes de MS7 (fichier : 10-classes-ms7-statistiques.pdf)

Entité	Attributs principaux	Rôle
EvenementRecu	id, type, source, payload, recuLe	Événement reçu de RabbitMQ (traçabilité)

Entité	Attributs principaux	Rôle
Indicateur	id, code, libelle, valeur, periode, calculeLe	Valeur agrégée sur une période
Rapport	id, titre, debut, fin, genereLe	Rapport regroupant des indicateurs

Relations. Les événements reçus sont agrégés en indicateurs (dépendance) ; un rapport regroupe un ou plusieurs indicateurs (1 → 1..*).

Énumérations. TypeEvenement : RENDEZ_VOUS_RESERVE, RESERVATION_CREEE, PRISE_CONFIRMEE, PRISE_OUBLIEE, VACCIN_ENREGISTRE, DON_ENREGISTRE, DEMANDE_SANG_PUBLIEE.

Règles de gestion.

- Chaque événement est enregistré une seule fois (identifiant d'événement) pour éviter les doubles comptages.
- Le tableau de bord est réservé au rôle Administrateur (403 sinon).

5. Interactions entre les services

5.1 Modes de communication

Tableau 6 — Modes de communication

Mode	Technologie	Quand l'utiliser
Synchrone	REST via OpenFeign, résolution du service par Eureka	Le service appelant a besoin de la réponse tout de suite pour continuer
Asynchrone	Événements publiés dans RabbitMQ	Pas besoin d'attendre la réponse ; découplage entre producteur et consommateurs

Les appels synchrones entre services passent directement par Eureka (nom logique du service) et non par la Gateway, qui reste le point d'entrée du frontend.

5.2 Interactions synchrones

Tableau 7 — Appels synchrones (OpenFeign)

Appelant → appelé	Appel	Données échangées	Justification
MS4 → MS1	`GET /api/medications/{id}`	Nom et dosage du médicament	Afficher immédiatement le traitement prescrit
MS5 → MS1	`GET /api/pharmacies/{id}`	Nom, adresse, téléphone de la pharmacie	La pharmacie appartient à MS1, MS5 ne la recopie pas
MS3 → MS4	`GET /api/treatments?patientId=`	Traitements en cours du patient	Utile au médecin pendant la consultation

5.3 Événements asynchrones

Les événements sont publiés dans un exchange RabbitMQ de type topic, `mediconnect.events`. Chaque consommateur possède sa propre file durable (par exemple `ms4.rappels` et `ms7.statistiques`), ce qui permet à plusieurs services de recevoir le même événement.

Tableau 8 — Catalogue des événements

Événement	Producteur	Consommateurs	Contenu	Usage
ReservationCreee	MS1	MS7	reservationId, medicamentId, pharmacieId, quantite	Statistiques des réservations
DemandeSangPubliee	MS2	MS7	demandeId, groupe, urgence, ville	Demandes par groupe sanguin
DonEnregistre	MS2	MS7	donId, groupe, volumeMl, date	Volume de dons collectés
RendezVousReserve	MS3	MS4, MS7	rendezVousId, patientId, medecinId, debut	Rappel la veille + statistiques
RendezVousAnnule	MS3	MS4, MS7	rendezVousId, patientId	Supprimer le rappel + statistiques
PriseConfirmee / PriseOubliee	MS4	MS7	priseId, traitementId, patientId, date	Indicateurs de suivi des traitements

Événement	Producteur	Consommateurs	Contenu	Usage
VaccinEnregistre	MS6	MS4, MS7	doseId, patientId, vaccin, prochaineEcheance	Rappel de la prochaine dose + couverture vaccinale

5.4 Comportement en cas de panne

Tableau 9 — Tolérance aux pannes

Situation	Comportement prévu
MS1 indisponible lors d'un appel de MS4	Délai d'attente court puis repli OpenFeign : le traitement est créé avec l'identifiant du médicament ; le nom s'affiche « indisponible ».
MS1 indisponible lors d'un appel de MS5	La garde est affichée avec la zone et les horaires ; le nom de la pharmacie s'affiche « indisponible ». Les urgences, propres à MS5, restent affichées.
MS4 indisponible lors d'un appel de MS3	La consultation s'affiche sans les traitements, avec le message « traitements indisponibles ». La réservation n'est jamais bloquée.
Consommateur (MS4 ou MS7) indisponible	Les événements restent dans les files durables de RabbitMQ et sont traités au redémarrage : aucun événement perdu.
RabbitMQ indisponible	L'opération métier est enregistrée dans la base du producteur ; l'événement est republié plus tard.

5.5 Exemple de scénario : réservation d'un rendez-vous

1. Le patient choisit un créneau dans Angular ; la requête `POST /api/appointments` (avec le JWT) arrive à la Gateway.
2. La Gateway vérifie le jeton puis route la requête vers MS3 grâce à Eureka.
3. MS3 vérifie que le créneau est libre, enregistre le rendez-vous dans sa base et répond `201 Created`.
4. MS3 publie l'événement `RendezVousReserve` dans RabbitMQ.
5. MS4 le consomme et programme un rappel la veille ; MS7 le consomme et met à jour les statistiques.

6. Diagramme d'architecture globale

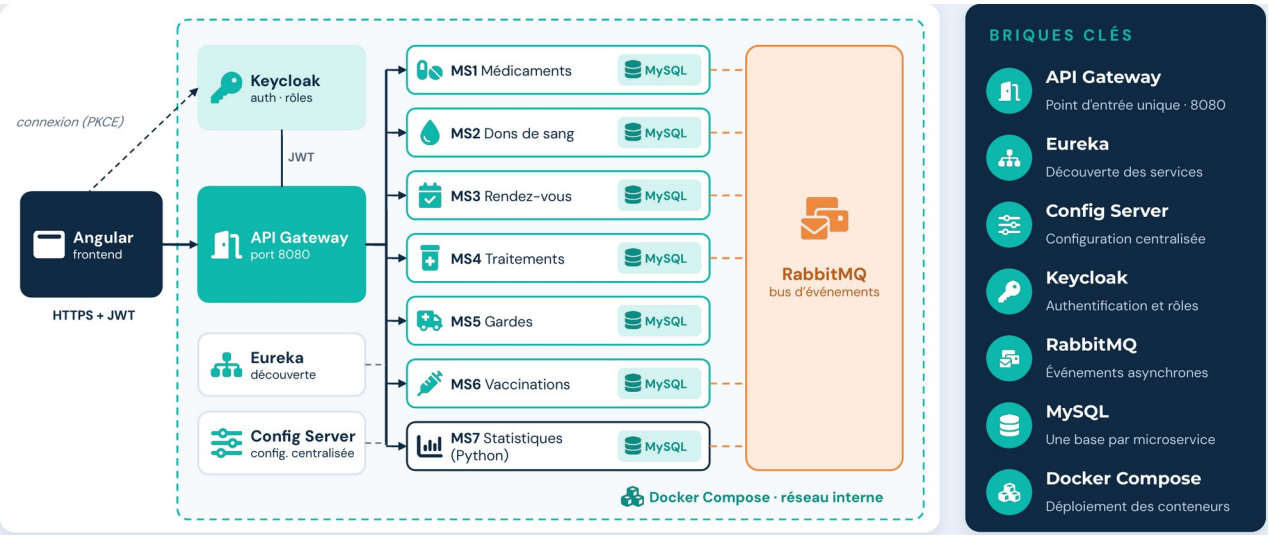


Figure 10 — Diagramme d'architecture globale (fichier : 03-architecture-globale.pdf)

6.1 Composants

Tableau 10 — Composants de l'architecture

Composant	Rôle	Technologie	Port
Frontend	Interface utilisateur, un module par domaine	Angular	4200
API Gateway	Point d'entrée unique, routage, CORS, routes protégées	Spring Cloud Gateway	8080
Eureka	Annuaire : chaque service s'y enregistre, la Gateway et OpenFeign le consultent	Spring Cloud Netflix Eureka	8761
Config Server	Configuration centralisée de tous les services	Spring Cloud Config	8888
Keycloak	Authentification, émission des JWT, gestion des rôles	Keycloak	8180
MS1 à MS6	Services métier	Spring Boot	8081–8086
MS7	Service commun d'analyse	Python · FastAPI	8087
Bases de données	Une base par microservice	MySQL	3306
Bus d'événements	Communication asynchrone	RabbitMQ	5672
Déploiement	Conteneurs et réseau interne	Docker Compose	—

6.2 Parcours d'une requête

1. L'utilisateur se connecte : Angular le redirige vers Keycloak, qui renvoie un jeton JWT.
2. Angular appelle la Gateway (port 8080) en ajoutant `Authorization: Bearer <JWT>`.
3. La Gateway vérifie le jeton puis interroge Eureka pour trouver une instance du service ciblé (`lb://nom-du-service`).
4. Le microservice vérifie à son tour le jeton et le rôle, traite la requête avec sa base MySQL et répond.
5. Si nécessaire, il appelle un autre service (OpenFeign) ou publie un événement (RabbitMQ).

6.3 Routage de l'API Gateway

Tableau 11 — Routes de la Gateway

Préfixe de route	Service cible
`/api/medications/**` · `/api/reservations/**`	`lb://ms1-medicaments`
`/api/donors/**` · `/api/blood-requests/**` · `/api/donations/**`	`lb://ms2-dons-sang`
`/api/doctors/**` · `/api/appointments/**`	`lb://ms3-rendez-vous`
`/api/treatments/**` · `/api/intakes/**`	`lb://ms4-traitements`
`/api/on-call/**`	`lb://ms5-gardes`
`/api/vaccines/**` · `/api/vaccination-records/**` · `/api/vaccinations/**`	`lb://ms6-vaccinations`
`/api/analytics/**`	`lb://ms7-statistiques`

6.4 Configuration centralisée et déploiement

Chaque service ne garde localement que son nom et l'adresse du Config Server. Le Config Server fournit le reste : port, connexion à sa base MySQL, adresse d'Eureka et de RabbitMQ, routes de la Gateway. Docker Compose lance l'ensemble des conteneurs sur un réseau interne : seule la Gateway (et Keycloak pour la connexion) est exposée à l'extérieur.

7. Autres éléments nécessaires à la compréhension de la conception

Cette partie regroupe les choix qui complètent la conception : la sécurité, les contrats d'API prévus pour chaque service et les technologies retenues.

7.1 Sécurité

7.1.1 Authentification

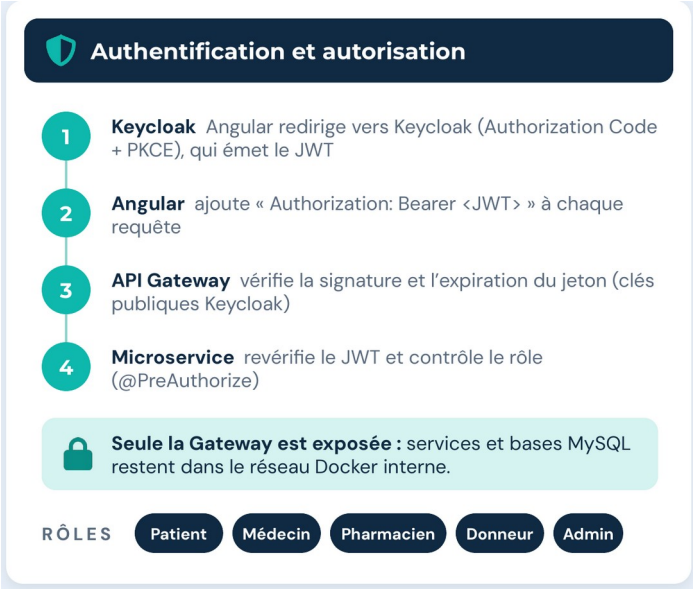


Figure 11 — Authentification et autorisation

1. **Keycloak** : Angular redirige l'utilisateur vers Keycloak (flux Authorization Code + PKCE, adapté à une application dans le navigateur), qui émet un JWT.
2. **Angular** ajoute `Authorization: Bearer <JWT>` à chaque requête.
3. **API Gateway** : vérifie la signature et l'expiration du jeton avec les clés publiques de Keycloak ; les requêtes sans jeton valide sont rejetées (401).
4. **Microservice** : revérifie le jeton (défense en profondeur) et contrôle le rôle avec `@PreAuthorize` (403 si le rôle est insuffisant).

7.1.2 Rôles et autorisations

Tableau 12 — Matrice rôles × microservices

Rôle	MS1	MS2	MS3	MS4	MS5	MS6	MS7
Patient	Rechercher, réserver	—	Prendre / annuler un RDV	Suivre, confirmer une prise	Consulter	Consulter son carnet	—
Médecin	Consulter	Publier une demande	Gérer créneaux, consultations	Prescrire	Consulter	Enregistrer une dose	—
Pharmacien	Gérer stocks, réservations	—	—	—	Déclarer les gardes	—	—
Donneur	—	S'inscrire, donner	—	—	Consulter	—	—

Rôle	MS1	MS2	MS3	MS4	MS5	MS6	MS7
Admin	Superviser	Superviser	Superviser	Superviser	Superviser	Superviser	Tableau de bord, rapports

7.1.3 Protection du réseau et des données

- Seule la Gateway est exposée : les microservices et les bases MySQL restent dans le réseau Docker interne.
- Un utilisateur MySQL par base, avec des droits limités à cette base.
- Les mots de passe et secrets sont fournis par variables d'environnement ou par le Config Server, jamais écrits dans le code.

7.2 Contrats d'API

7.2.1 Conventions

- Un contrat **Swagger / OpenAPI** par service, figé après la séance 4 (fichiers `*.openapi.yaml` dans la documentation, puis générés par springdoc).
- Ressources au pluriel, méthodes HTTP standard : `GET` (lecture), `POST` (création), `PATCH` (modification partielle), `DELETE` (suppression).
- Codes de réponse : `200` OK, `201 Created` avec l'en-tête `Location`, `204 No Content`, `400` données invalides, `401` non authentifié, `403` rôle insuffisant, `404` introuvable, `409` conflit avec une règle métier.
- Les erreurs sont renvoyées au format **Problem Details** (RFC 7807).

7.2.2 Principaux endpoints par service

MS1 — Médicaments & pharmacies

Méthode	URI	Description	Codes
GET	/api/medications?nom=	Rechercher des médicaments par nom	200 · 400
GET	/api/medications/{id}/availability	Disponibilité d'un médicament par pharmacie	200 · 404
POST	/api/reservations	Réserver un médicament	201 + Location · 400 · 409
PATCH	/api/reservations/{id}	Changer le statut d'une réservation	200 · 404 · 409

Endpoints internes appelés par OpenFeign : `GET /api/medications/{id}` (par MS4) et `GET /api/pharmacies/{id}` (par MS5).

MS2 — Dons de sang

Méthode	URI	Description	Codes
POST	/api/donors	Inscrire un donneur	201 + Location · 400 · 409
POST	/api/blood-requests	Publier une demande de sang	201 + Location · 400
GET	/api/blood-requests?groupe=&ville=	Lister les demandes ouvertes	200 · 400

Méthode	URI	Description	Codes
POST	/api/donations	Enregistrer un don	201 + Location · 400 · 409

MS3 — Rendez-vous médicaux

Méthode	URI	Description	Codes
GET	/api/doctors?specialite=&ville=	Rechercher des médecins	200 · 400
GET	/api/doctors/{id}/slots	Créneaux disponibles d'un médecin	200 · 404
POST	/api/appointments	Réserver un rendez-vous	201 + Location · 400 · 409
PATCH	/api/appointments/{id}/cancel	Annuler un rendez-vous	200 · 404 · 409

MS4 — Traitements & rappels

Méthode	URI	Description	Codes
POST	/api/treatments	Créer un traitement	201 + Location · 400
GET	/api/treatments?patientId=	Traitements d'un patient	200 · 400
GET	/api/treatments/{id}/intakes	Prises d'un traitement	200 · 404
PATCH	/api/intakes/{id}	Confirmer ou marquer une prise oubliée	200 · 404 · 409

`GET /api/treatments?patientId=` est aussi appelé en synchrone par MS3.

MS5 — Gardes & urgences

Méthode	URI	Description	Codes
GET	/api/on-call/pharmacies?zone=&date=	Pharmacies de garde	200 · 400
GET	/api/on-call/emergency-services?zone=	Services d'urgence d'une zone	200 · 400
POST	/api/on-call/shifts	Déclarer une garde	201 + Location · 400 · 409
DELETE	/api/on-call/shifts/{id}	Supprimer une garde	204 · 404

MS6 — Vaccinations

Méthode	URI	Description	Codes
GET	/api/vaccines	Lister les vaccins	200
GET	/api/vaccination-records/{id}	Consulter un carnet	200 · 404
POST	/api/vaccinations	Enregistrer une dose	201 + Location · 400 · 409
GET	/api/vaccination-records/{id}/due	Prochaines échéances	200 · 404

MS7 — Analyse & statistiques (commun)

Méthode	URI	Description	Codes
GET	/api/analytics/dashboard	Tableau de bord	200 · 403
GET	/api/analytics/indicators?periode=	Indicateurs d'une période	200 · 400
POST	/api/analytics/reports	Générer un rapport	201 + Location · 400
GET	/api/analytics/reports/{id}	Consulter un rapport	200 · 404

7.3 Technologies choisies

Tableau 13 — Stack technique

Couche	Technologie	Justification
Présentation	Angular	Framework imposé ; un module par domaine métier ; intégration Keycloak (PKCE)
Accès	Spring Cloud Gateway	Point d'entrée unique, routage dynamique `lb://`, filtres et CORS
Sécurité	Keycloak + JWT	SSO, gestion centralisée des utilisateurs et des rôles, standard OAuth2 / OpenID Connect
Services métier	Spring Boot	Stack imposée ; REST, JPA, validation, OpenFeign, documentation Swagger
Service commun	Python · FastAPI	Technologie avancée ; API rapide à écrire, adaptée au traitement de données
Infrastructure	Eureka · Config Server	Découverte dynamique des instances ; configuration centralisée
Échanges	RabbitMQ	Événements asynchrones, files durables, découplage et résilience
Données	MySQL	Données relationnelles, transactions ACID, une base par service
Déploiement	Docker Compose	Environnement reproductible pour toute l'équipe